



National University
of computer and emerging sciences

DEPARTMENT OF COMPUTER SCIENCE

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING
SCIENCES.

(NUCES)



REPORT

By

Shahram Shafiq – 24i-2541



Course Title:	Artificial Intelligence
Semester:	4 th - Spring
Date of Submission:	16 th February 2026
Instructor Name:	Ms. Marium Hida

Dungeon Quest - Dragon's Lair

Course: Artificial Intelligence

Student Name: Shahram Shafiq

Date: 16 February

1. OVERVIEW

This project is a game based GUI which implements multiple path finding algorithms like the : BFS, DFS, UCS, A* (Manhattan), and A* (Euclidean) and also adds a validation check and goes through walls and obstacles.

2. PROGRAM ARCHITECTURE

2.1 Core Components

Data Structures (Manual Implementation):

- Grid Representation: 2D array based grid in which 0 is free, 1 is block
- Queue (BFS): Made using an array with front and rear pointers
- Stack (DFS): Fixed-size array with top pointer
- Priority Queue (UCS/A):* Array-based with linear search for minimum

GUI Components:

- Header with game statistics (HP, Coins, Explored tiles)
- Scrollable control panel with algorithm selection



- Interactive canvas grid with sprite-based visualization
- Real-time animation system

3. ALGORITHM IMPLEMENTATIONS

3.1 BREADTH-FIRST SEARCH (BFS)

Steps Involved:

1. Initialize Data Structures
 - a. Create visited array (all False)
 - b. Create parent tracking array
 - c. Create manual queue using fixed-size array
 - d. Set front = 0, rear = 0
2. Enqueue Start Position
 - a. Add start position to queue
 - b. Mark start as visited
 - c. Increment rear pointer
3. BFS Main Loop
 - a. While queue not empty (front $\neq$ rear):
 - i. Dequeue node from front
 - ii. Check if goal reached
 - iii. Explore 4 neighbors (up, down, left, right)
 - iv. For each valid, unvisited neighbor:
 1. Mark as visited
 2. Store parent relationship

3. Enqueue to queue
4. Visualize as explored

4. Path Reconstruction

- a. Start from goal position
- b. Trace back through parent pointers
- c. Reverse path to get start-to-goal order
- d. Animate hero movement along path

Key Features:

- Guarantees shortest path in number of steps
- FIFO exploration ensures level-by-level search
- Marks nodes as visited when enqueued

3.2 DEPTH-FIRST SEARCH (DFS)

Steps Involved:

1. Initialize Stack
 - a. Create visited array
 - b. Create parent tracking array
 - c. Create manual stack using fixed-size array
 - d. Set top = -1 (empty stack)
2. Push Start Position
 - a. Push start to stack (top++)
 - b. Mark start as visited
3. DFS Main Loop



- a. While stack not empty ($\text{top} \neq -1$):
 - i. Pop node from top
 - ii. Check if goal reached
 - iii. Explore 4 neighbors
 - iv. For each valid, unvisited neighbor:
 1. Mark as visited immediately
 2. Store parent
 3. Push to stack
 4. Visualize exploration
4. Path Reconstruction
 - a. Trace back from goal using parent pointers
 - b. Reverse path
 - c. Animate hero movement

Key Features:

- LIFO exploration (explores one branch deeply)
- Does not guarantee shortest path
- Uses less memory than BFS in practice

3.3 UNIFORM COST SEARCH (UCS)

Steps Involved:

1. Initialize Cost System
 - a. Generate random costs (1-10) for each free cell
 - b. Create visited array and parent array



- c. Create manual priority queue (frontier)
- d. Set front = 0, rear = 0
2. Add Start Node
 - a. Insert start with cost = 0
 - b. Mark as visited
3. UCS Main Loop
 - a. While frontier not empty:
 - i. Find minimum cost node (linear search)
 - ii. Remove node from frontier
 - iii. Check if goal reached
 - iv. For each neighbor:
 1. Calculate $\text{new_cost} = \text{current_cost} + \text{cell_cost}$
 2. If not visited:
 - a. Mark visited
 - b. Store parent
 - c. Add to frontier with new cost
 3. Visualize exploration
4. Cost Calculation & Path Display
 - a. Reconstruct path from goal to start
 - b. Sum up costs of cells in path
 - c. Display total cost in GUI with \$ sign
 - d. Animate hero collecting coins

Key Features:

- Always finds least-cost path
- Uses cell costs (not just distance)
- Priority queue ordered by cumulative cost

3.4 A SEARCH (Manhattan Distance)*

Steps Involved:

1. Initialize A Structures*
 - a. Create g_cost array (cost from start)
 - b. Create visited and closed arrays
 - c. Create parent array
 - d. Create open set (priority queue)
2. Calculate Initial Heuristic
 - a. Manhattan Distance: $h = |x_1 - x_2| + |y_1 - y_2|$
 - b. Calculate $f = g + h$ for start node
 - c. Add start to open set
3. A Main Loop*
 - a. While open set not empty:
 - i. Find node with minimum f-value
 - ii. If tie, choose lower g-value
 - iii. Remove from open set
 - iv. Add to closed set
 - v. Check if goal reached
 - vi. For each neighbor:



1. Skip if in closed set
2. Calculate: $\text{new_g} = g + 1$
3. Calculate: $\text{new_h} = \text{Manhattan distance to goal}$
4. Calculate: $\text{new_f} = \text{new_g} + \text{new_h}$
5. If not visited OR cheaper path found:
 - a. Update g_cost , parent
 - b. Add to open set
6. Visualize exploration

4. Path Display

- a. Reconstruct optimal path
- b. Display step count
- c. Animate hero movement

Key Features:

- Informed search using heuristic
- Manhattan heuristic: sum of horizontal and vertical distances
- Admissible (never overestimates)
- Guarantees optimal path

3.5 A SEARCH (Euclidean Distance)*

Steps Involved:

1. Same as A Manhattan* except heuristic calculation
2. Euclidean Heuristic
 - a. $h = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

- b. Straight-line distance to goal
3. Rest of algorithm identical to Manhattan

Key Features:

- Also admissible heuristic
- Often explores fewer nodes than Manhattan
- Better for scenarios allowing diagonal movement (though our grid doesn't)

4. GUI FEATURES

4.1 Interactive Elements

1. Grid Builder
 - a. Click and drag to place walls (brick sprites)
 - b. Click again to remove walls
 - c. Place hero and dragon
2. Control Panel
 - a. Scrollable interface for all controls
 - b. Grid size adjustment (10×10 to 50×50)
 - c. Algorithm selection via radio buttons
 - d. Pattern generators (Maze, Random)
3. Action Buttons
 - a. Begin Quest: Runs selected algorithm
 - b. Clear Path: Removes exploration visualization
 - c. Reset Grid: Clears entire dungeon
4. Real-time Statistics



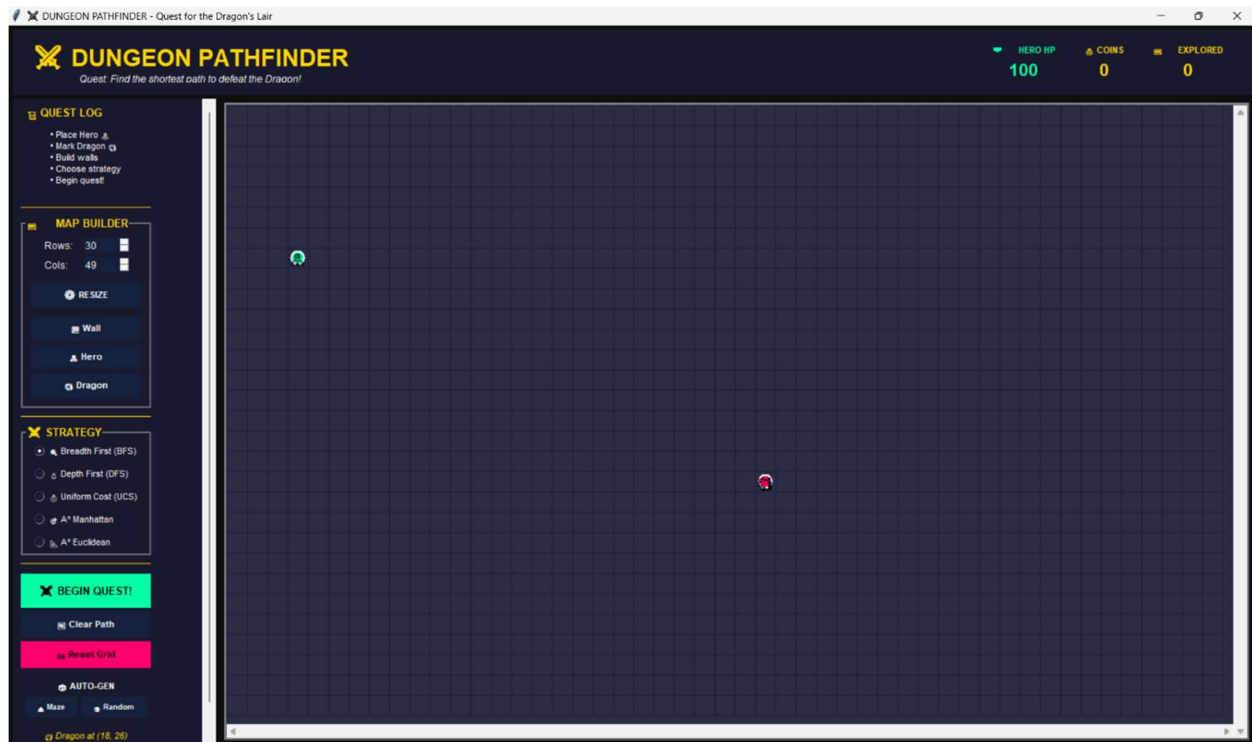
- a. Hero HP: 100
- b. Coins collected during journey
- c. Tiles explored by algorithm
- d. Quest cost (for UCS/A*)

4.2 Visualization

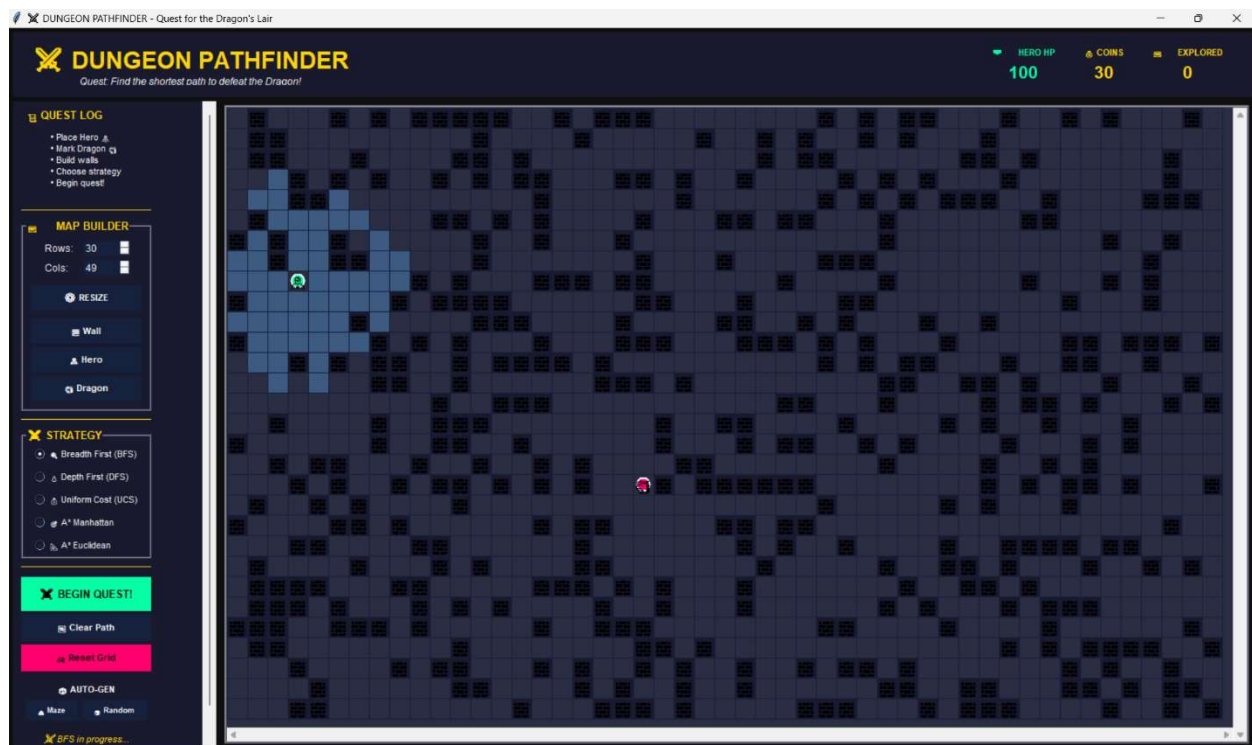
- Floor: Dark grey dungeon tiles
- Walls: Dark blocks with brick emoji
- Hero: Green circle with emoji
- Dragon: Red circle with emoji
- Explored: Blue tiles showing algorithm progress
- Path: Golden trail with coin animations
- Smooth Animation: Hero walks step-by-step to dragon

5. SCREENSHOTS

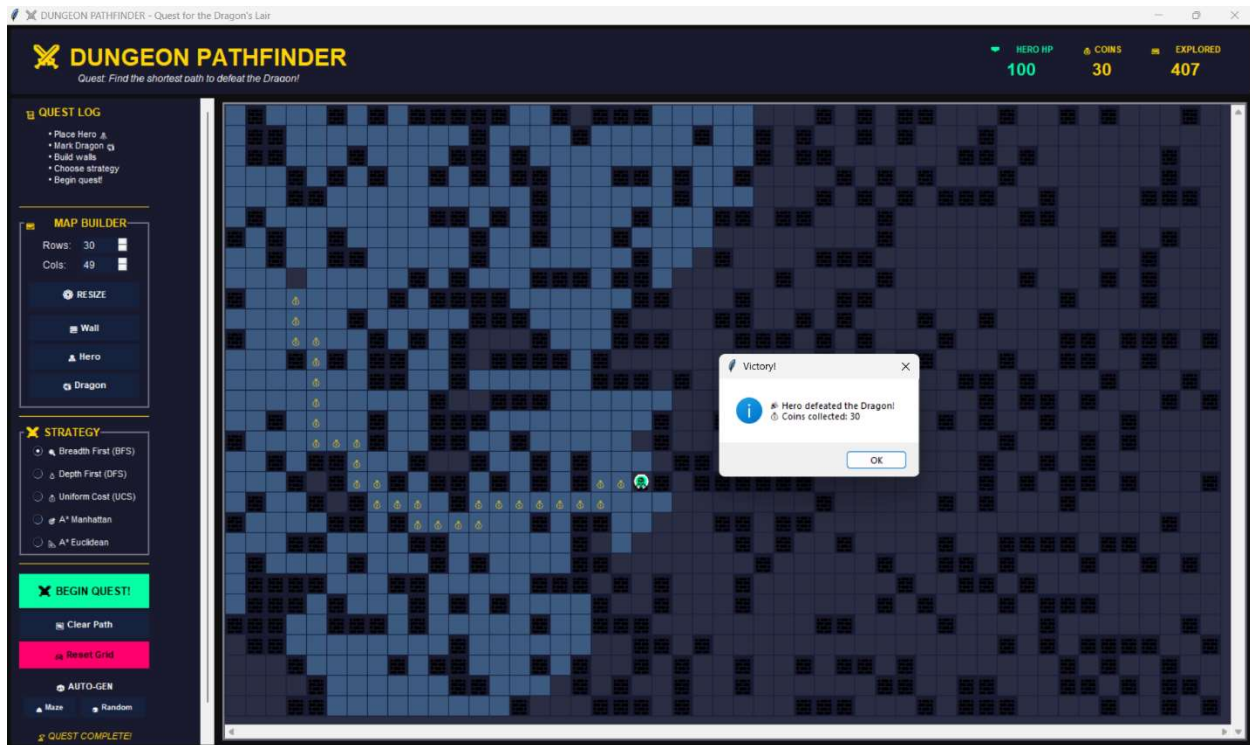
Screenshot 1: Initial grid with hero and dragon placed



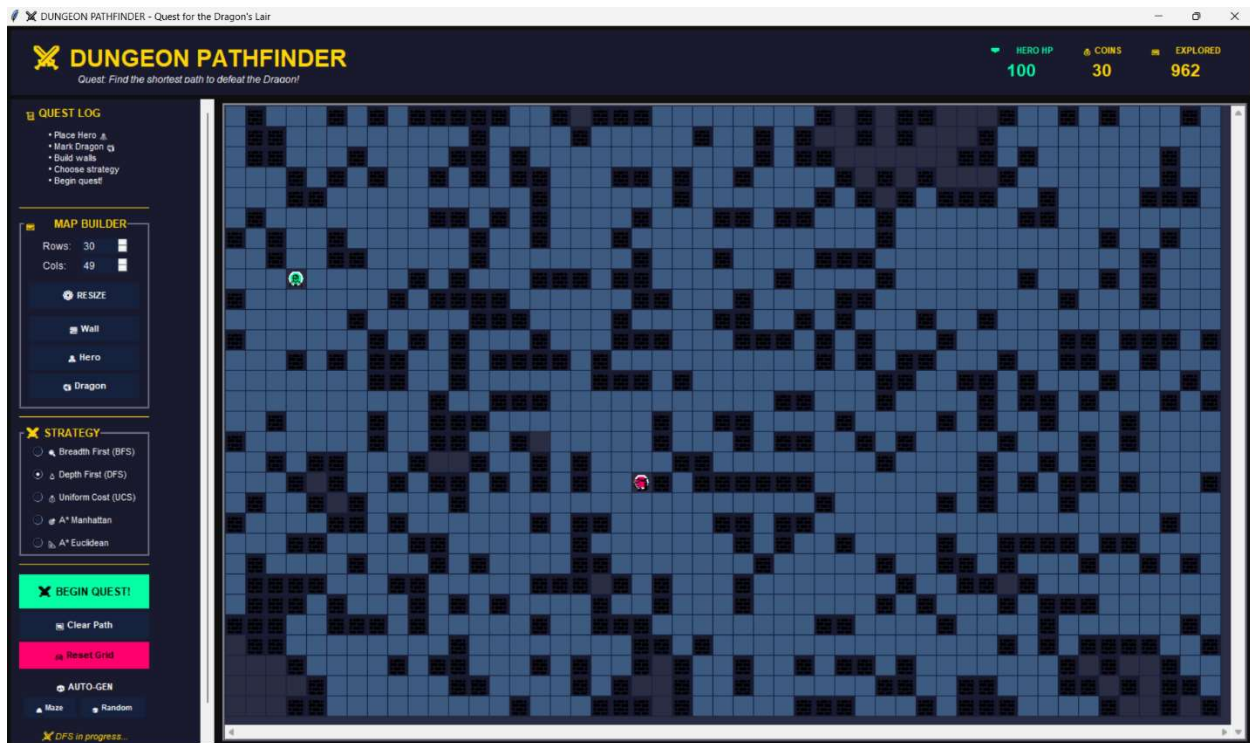
Screenshot 2: BFS exploration pattern (blue tiles)



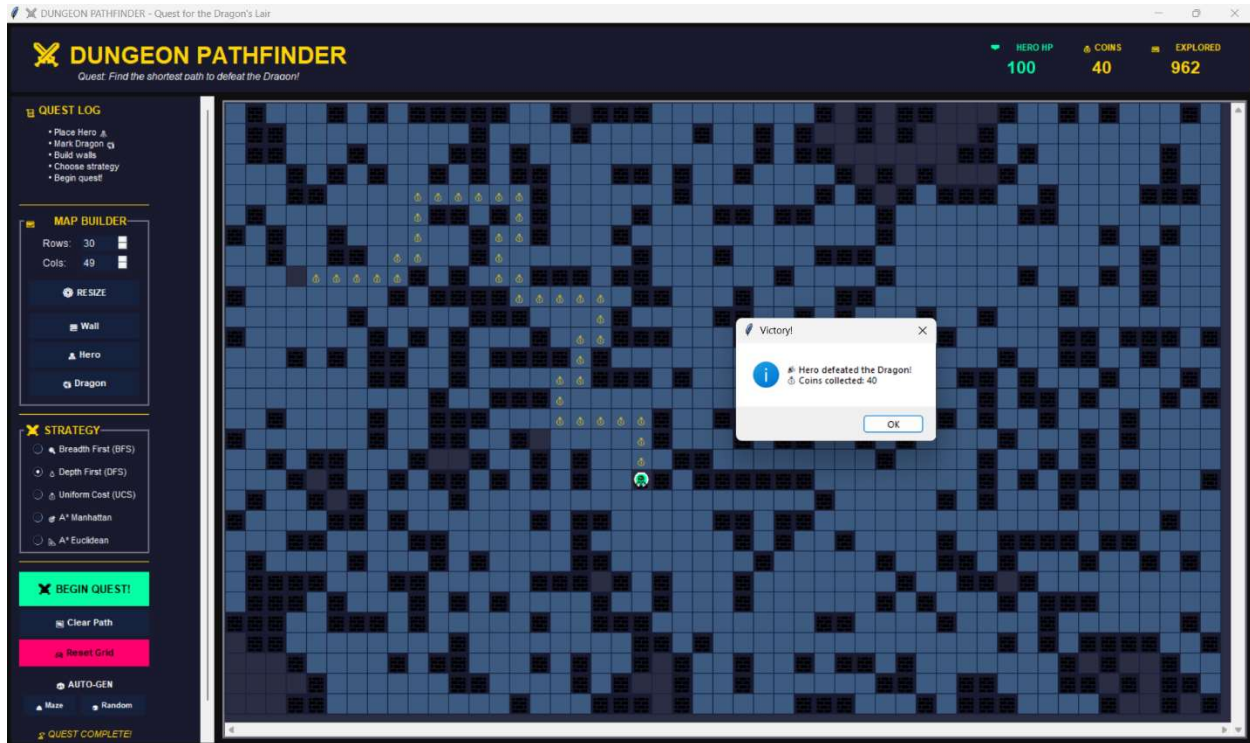
Screenshot 3: BFS final path (golden trail with coins)



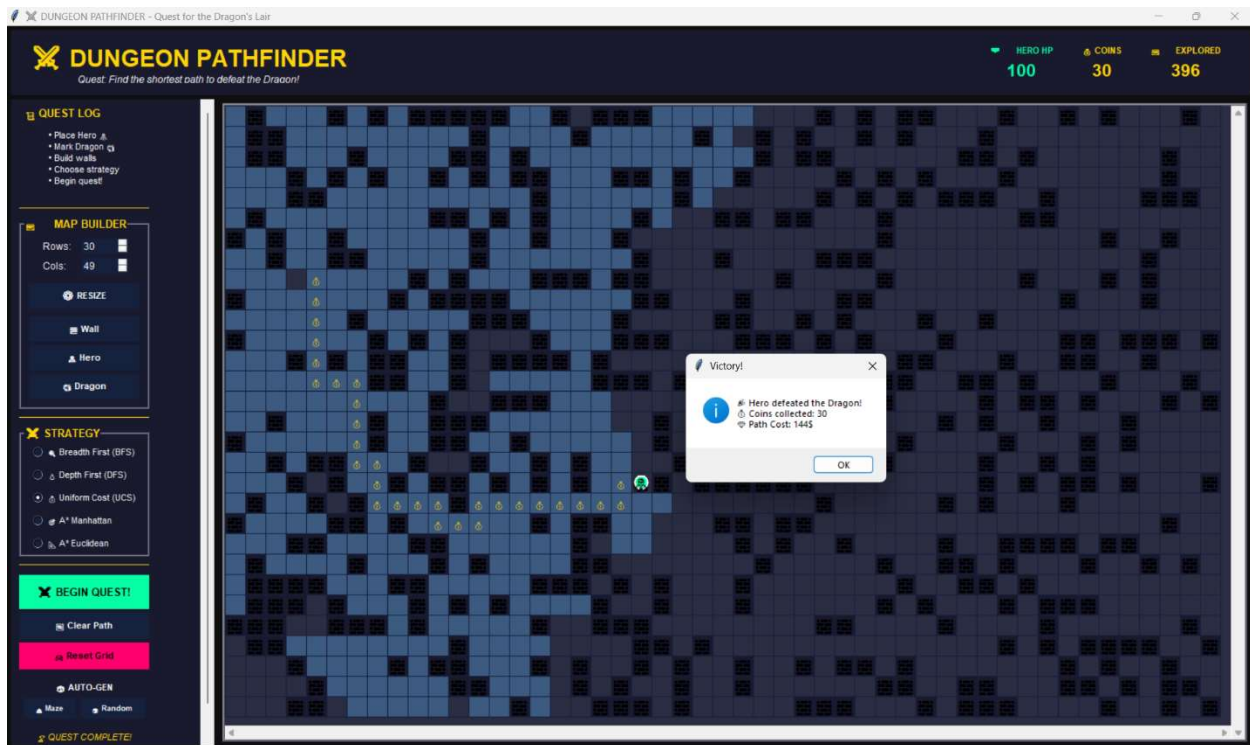
Screenshot 4: DFS exploration pattern



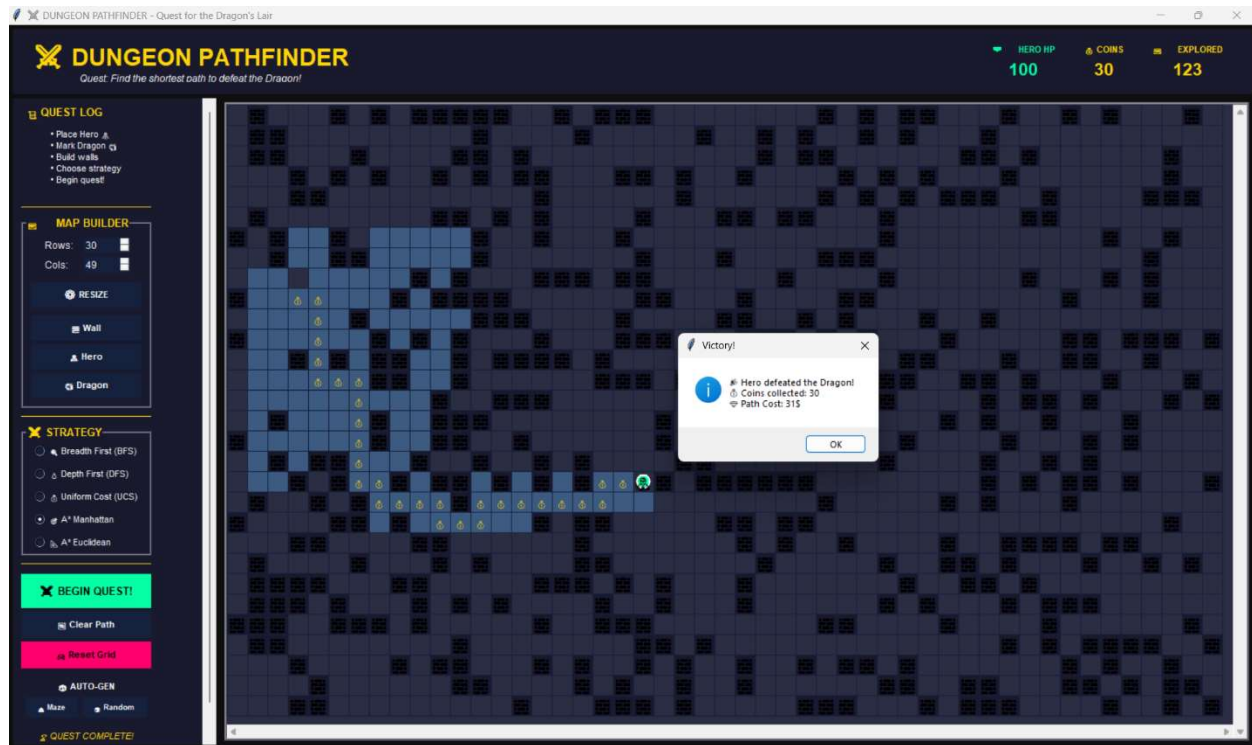
Screenshot 5: DFS final path



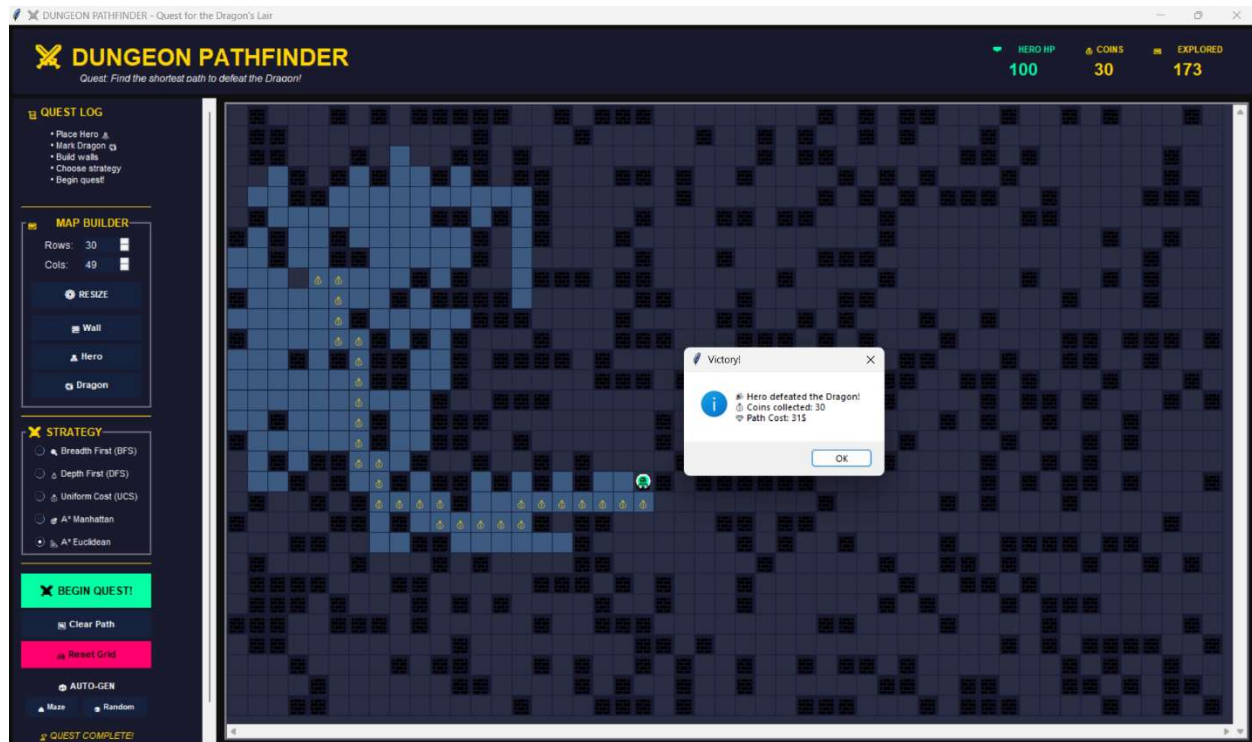
Screenshot 6: UCS with cost display



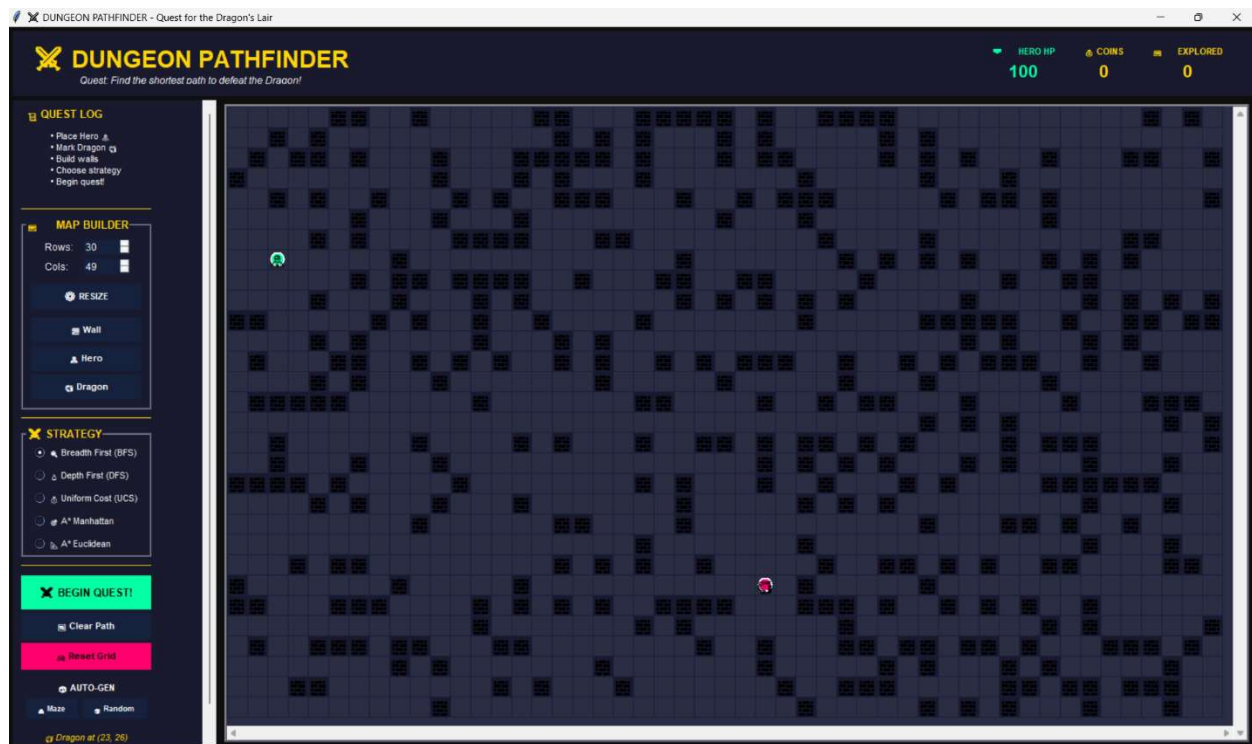
Screenshot 7: A* Manhattan exploration



Screenshot 8: A* Euclidean result



Screenshot 9: Full GUI overview





6. COMPLEXITY ANALYSIS

6.1 Time Complexity

Algorithm	Time Complexity	Explanation
BFS	$O(V + E) = O(R \times C)$	Visits each cell once. R = rows, C = columns. Each cell checks 4 neighbors.
DFS	$O(V + E) = O(R \times C)$	Same as BFS - visits each cell once in worst case.
UCS	$O(V^2) = O((R \times C)^2)$	Due to linear search for minimum in frontier. Each of V nodes may require O(V) search. With proper heap: $O((V+E) \log V)$
A* (Both)	$O(V^2) = O((R \times C)^2)$	Same as UCS - linear search for minimum f-value. With proper heap: $O((V+E) \log V)$. Heuristic reduces nodes explored in practice.

Notes:

- V = number of vertices (cells) = rows $\times$ cols
- E = number of edges = $4V$ (each cell has up to 4 neighbors)
- Manual implementation uses linear search instead of heap

- A* typically explores fewer nodes than UCS due to heuristic guidance

6.2 Space Complexity

Algorithm	Space Complexity	Explanation
BFS	$O(R \times C)$	Queue ($R \times C$), visited array ($R \times C$), parent array ($R \times C$). All proportional to grid size.
DFS	$O(R \times C)$	Stack ($R \times C$), visited array ($R \times C$), parent array ($R \times C$).
UCS	$O(R \times C)$	Frontier ($R \times C$), visited ($R \times C$), parent ($R \times C$), grid_cost ($R \times C$). Multiple arrays but all $O(R \times C)$.
A (Both)*	$O(R \times C)$	Open set ($R \times C$), closed set ($R \times C$), g_cost ($R \times C$), parent ($R \times C$), visited ($R \times C$). All linear in grid size.

Implementation Details:

- All data structures use fixed-size arrays
- No dynamic resizing (no `.append()`, `.pop()`)
- Manual index pointers for queue/stack operations
- Arrays pre-allocated to maximum grid size (50×50)

7. KEY IMPLEMENTATION DETAILS

7.1 Manual Data Structures

Queue (BFS):

python

```
queue = [None] * (rows * cols) # Fixed size
```

```
front = 0 # Dequeue position
```



```
rear = 0 # Enqueue position
```

```
# Enqueue: queue[rear] = value; rear += 1
```

```
# Dequeue: value = queue[front]; front += 1
```

Stack (DFS):

python

```
stack = [None] * (rows * cols)
```

```
top = -1 # Stack pointer
```

```
# Push: top += 1; stack[top] = value
```

```
# Pop: value = stack[top]; top -= 1
```

Priority Queue (UCS/A):*

python

```
frontier = [None] * (rows * cols)
```

```
# Extract-Min: Linear search O(n) for minimum
```

```
# Insert: frontier[rear] = value; rear += 1
```

7.2 Grid Representation

- Internal: 2D array with 0 and 1 to represent free and blocked
- No other encodings as per requirements
- 4-directional movement only with diagonals not handled

8. TESTING & VALIDATION

8.1 Test Cases

1. Simple Path: Hero and dragon adjacent
2. Blocked Path: Wall between hero and dragon



3. No Path: Dragon completely surrounded
4. Large Grid: 50×50 with maze
5. Multiple Paths: Compare algorithms on same grid

8.2 Correctness Verification

BFS always finds shortest path (steps)

DFS finds valid path (not necessarily shortest)

UCS finds least-cost path

A* finds optimal path with both heuristics

All algorithms handle unreachable goals correctly

Blocked cells never expanded

9. FEATURES BEYOND REQUIREMENTS

1. Game-Style Interface: dungeon theme
2. Sprite-Based Graphics: Emojis for hero, dragon, walls
3. Animated Hero Movement: Step-by-step walking animation
4. Coin Collection System: Visual feedback on path
5. Real-time Statistics: Live updates during execution
6. Pattern Generators: Maze and random wall creation
7. Scrollable Panel: Fits all controls on screen
8. Cost Display: Shows path cost with \$ for UCS/A*

10. CONCLUSION



National University
of computer and emerging sciences

This project successfully implements all required pathfinding algorithms with proper manual data structures and comprehensive visualization and uses a game based interface